

Guidance Navigation Control and Simulation for VTOL vehicles

Pablo Plata

Aeronautical and Astronautical Engineering
Purdue University
PSP - Active Controls

January 23, 2026

Contents

1 Abstract **2**

2 Simulation **3**
2.1 Equations of Motion . . . 3
2.1.1 Conventions and Frame Definitions . . . 3
2.1.2 Dynamics Derivation . . . 3
2.2 Actuator Models . . . 5
2.3 Wind Disturbance Models . . . 6
2.4 Sensor Models . . . 6
2.4.1 IMU . . . 6
2.4.2 GPS . . . 7

3 Estimation and Filtering **8**
3.1 Multiplicative Extended Kalman Filter [M-EKF] . . . 8
3.1.1 Motivation . . . 8
3.1.2 Error-State Formulation . . . 9
3.1.3 Multiplicative Extended Kalman Filter Formulation . . . 13
3.1.4 Onboard Application . . . 15
3.1.5 Observability Analysis . . . 15
3.2 Digital Filter Sequence [DFS] . . . 18
3.2.1 Motivation . . . 18
3.2.2 Analog and Discrete Formulation . . . 19

4 Controller **22**
4.1 Controller Iteration History . . . 22
4.2 Control Law Formulation . . . 22

5 Simulation Outputs **27**

Chapter 1

Abstract

This document covers the development, testing, and iteration process of a GNC & Simulation workflow for a vertical take-off and landing vehicle named TOAD. This vehicle is being developed by the Active Controls sub-team of the Purdue Space Program (PSP) chapter of SEDS. This liquid powered vehicle is designed to fly to 50 meters, hover within a 1 meter side cube target, and safely land back in a designated landing zone in compliance with the Collegiate Propulsive Lander Challenge (CPLC) competition. To safely perform an iterative design process for this architecture, a testbed vehicle powered by an electric powered fan was developed. This prototype vehicle is named ASTRAv2. The chosen architecture includes a Multiplicative Extended Kalman Filter [M-EKF] for state estimation, a digital filter sequence for sensor measurements, and a 3 loop cascaded controller with an LQRi stage for attitude tracking. This framework is then tested in our 6 degree of freedom simulation framework in MATLAB / Simulink which includes second order sensor models, wind and turbulence modeling, and parameter uncertainty.
This document will focus on the application of these controls to the ASTRAv2 prototype vehicle.

Chapter 2

Simulation

2.1 Equations of Motion

The creation of an accurate dynamics model for ASTRAv2 is crucial to the success of the testing campaign. The reliability of this model is crucial to a proper trajectory generation method, a good gain matrix generation, and proper Kalman Filter propagation steps. A good dynamics model augmented with disturbance and actuator models also helps validate all software aspects before flight, reducing risk and helping identify potential issues. This paper covers the derivation of ASTRA's equations of motion (EoM's), the Linear Quadratic Regulator (LQR) generation and the respective linearization, as well as the conventions used for the vehicle.

2.1.1 Conventions and Frame Definitions

ASTRA v2 will use an earth frame oriented as North, West, Up (NWU). This frame follows a right-hand rule convention. For the local frame, our Z-axis will point upward, and our other axis shall be aligned with the gimbal servo-actuators. As will be further discussed in this paper, ASTRA v2 will use a quaternion for it's attitude representation. This quaternion will represent a transformation from the body frame to the earth frame. We will refer to it as a body-frame quaternion. This quaternion will also use the scalar first convention for quaternion representations.

2.1.2 Dynamics Derivation

ASTRA v2 will use a twelve dimensional state vector for it's controls. This vector is defined as follows:

$$x = \begin{bmatrix} q \\ r \\ v \\ \omega \end{bmatrix} \tag{2.1}$$

where q is the attitude quaternion, r represents our three-dimensional position vector, v the three-dimensional velocities, and ω our body-frame angular rates. ASTRA v2 uses a two-axis gimbal

system, as well as contra-rotating propellers that provide thrust and torque control. As such, our input vector u is defined as:

$$u = \begin{bmatrix} \theta \\ \phi \\ T \\ \tau_{roll} \end{bmatrix} \tag{2.2}$$

where T is a total thrust command, θ and ϕ are our gimbal angles, and τ_{roll} is a torque command about the vehicle z-axis.
Our goal is to obtain a function $f(x, u)$ such that $\dot{x} = f(x, u)$. We first define the direction cosine matrix (DCM) from the Earth (inertial) frame to the Body frame as:

$$C_i^b = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1 q_2 + q_0 q_3) & 2(q_1 q_3 - q_0 q_2) \\ 2(q_1 q_2 - q_0 q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2 q_3 + q_0 q_1) \\ 2(q_1 q_3 + q_0 q_2) & 2(q_2 q_3 - q_0 q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix} \tag{2.3}$$

We also note that the transformation from the body frame to the inertial frame is the transpose of the transformation above: $C_b^i = (C_i^b)^T$. We now define the body-frame thrust vector T_b as:

$$T_b = T \begin{bmatrix} \cos(\theta)\sin(\phi) \\ -\sin(\theta) \\ \cos(\theta)\cos(\phi) \end{bmatrix} \tag{2.4}$$

We can now find the net force in the inertial frame:

$$F_i = C_b^i \cdot T_b - \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} \tag{2.5}$$

where m is the vehicle mass. We can now define:

$$\dot{r} = v \tag{2.6}$$

$$\dot{v} = \frac{F_i}{m} \tag{2.7}$$

Using quaternion kinematics, we note that:

$$\dot{q} = \frac{1}{2} M(q) \begin{bmatrix} 0 \\ \omega \end{bmatrix} \tag{2.8}$$

in which:

$$M(q) = \begin{bmatrix} q_0 & -q_1 & -q_2 & -q_3 \\ q_1 & q_0 & -q_3 & q_2 \\ q_2 & q_3 & q_0 & -q_1 \\ q_3 & q_2 & q_1 & q_0 \end{bmatrix} \tag{2.9}$$

We now find the body frame torque vector τ_b:

$$\tau_b = \vec{r}_{_{TB}} \times \vec{T}_b + \begin{bmatrix} 0 \\ 0 \\ \tau_{roll} \end{bmatrix} \tag{2.10}$$

where $\vec{r}_{_{TB}} = [\delta\bar{x}, \delta\bar{y}, r_{_{TB}}]^T$ represents the lever-arm vector, from the point of application of the force (engine) to the center of mass, $r_{_{TB}}$ is the vertical distance between the points, and $\delta\bar{x}, \delta\bar{y}$ are small components meant to represent small offsets in the CoM from the z-axis. Using Euler's Rigid Body equation it follows that:

$$\dot{\omega} = J^{-1}(\tau_b - \omega \times (J\omega)) \tag{2.11}$$

In summation, our dynamics are represented by the following system of differential equations:

$$\dot{x} = f(x, u) = \begin{bmatrix} \frac{1}{2}M(q) \begin{bmatrix} 0 \\ \omega \end{bmatrix} \\ v \\ \frac{1}{m}\boldsymbol{F_i} \\ J^{-1}(\tau_b - \omega \times (J\omega)) \end{bmatrix}$$

2.2 Actuator Models

The actuator models for this system are fairly simple. both thrust and torque inputs have simulated responses that follow a first order lag model:

$$\frac{dF}{dt} = \frac{1}{\tau_F} \cdot (F_{cmd} - F_{true}) \tag{2.12}$$

A quadratic loss coefficient $\delta F = k \cdot \tau_{roll}^2$ is then subtracted from the true thrust command depending on the torque command issued. This is meant to capture unmodeled aerodynamic losses from the differential thrust used to produce roll torques.
The gimbal is composed of two servo actuators, which we model as a lightly coupled system to represent small undesired crosstalk.

$$\frac{d\theta}{dt} = \frac{1}{\tau_\theta} \cdot (\theta_{cmd} - \theta_{true}) + K_{\phi\to\theta} \cdot \phi_{true} \tag{2.13}$$

$$\frac{d\phi}{dt} = \frac{1}{\tau_\phi} \cdot (\phi_{cmd} - \phi_{true}) + K_{\theta\to\phi} \cdot \theta_{true} \tag{2.14}$$

2.3 Wind Disturbance Models

To validate the robustness of the control laws against atmospheric uncertainties, the simulation environment incorporates a multi-layered wind disturbance model. The total wind vector acting on the vehicle is composed of a deterministic mean wind component, a wind gust field sample, and a specific Gauss-Markov process for additional disturbance modeling. The baseline mean wind field is generated using the Horizontal Wind Model 14 (HWM14) available in MATLAB. This model provides wind velocity estimates based on the day of the year and the simulation time. Superimposed on this mean field is the Von Karman Wind Turbulence Model, which generates continuous gusts based on the vehicle's altitude and velocity to simulate realistic atmospheric instability. Both of these models are available for use in Simulink via the Aerospace Toolbox. In addition to standard atmospheric models, a First-Order Gauss-Markov Process (GMP) is utilized to inject persistent, correlated noise into the system. The propagation of this disturbance is governed by:

$$\epsilon_k = \beta\epsilon_{k-1} + \sqrt{1-\beta^2} \cdot \eta \tag{2.15}$$

where η represents a standard normal random variable ($\eta \sim \mathcal{N}(0,1)$) scaled by a covariance factor. The correlation coefficient β is determined by the simulation time step Δt and a decay time constant τ, set to 10 seconds, via $\beta = e^{-\Delta t/\tau}$. Finally, the output of this process is passed through an Exponential Moving Average (EMA) filter to attenuate high-frequency noise before being applied to the vehicle dynamics.

2.4 Sensor Models

To properly test all systems involved in the flight software in simulation, we cannot assume we have access to perfect full-state feedback. Thus, we need to transform our "real" plant outputs into disturbed sensor models that can be fed into our nonlinear estimator. The motivation behind such an estimator will be discussed later. In real life, sensors will drift, have noise, all micro electronic mechanical sensors (MEMS) also have natural frequencies attributing them second order dynamics too. ASTRAv2 and TOAD will be equipped with at most 4 sensors. These include an accelerometer, a gyroscope, a magnetometer, and a GPS with real time kinematics (RTK) corrections, allowing for centimeter level precision.

2.4.1 IMU

Our IMU models leverage tools included in the Aerospace Toolbox for MATLAB / Simulink to create second-order accelerometer and gyroscope models. Our flight computer intends to use an ICM-40609D. The models available in the toolbox include second order dynamics, and can be expanded to model virtual forces measured from the accelerometer being away from the center of mass via:

$$\bar{a}_{IMU} = \bar{a}_{true} + \bar{\omega}_b \times (\bar{\omega}_b \times \bar{r}) + \dot{\bar{\omega}}_b \times \bar{r} + \bar{g}$$

Both the gyroscope model and the accelerometer model then get white noise added to them, along with a slowly drifting bias, modeled by a random walk. Finally, test data showed a clear relationship between thrust percentage and vibrational frequency recorded in the IMU (See Figure)

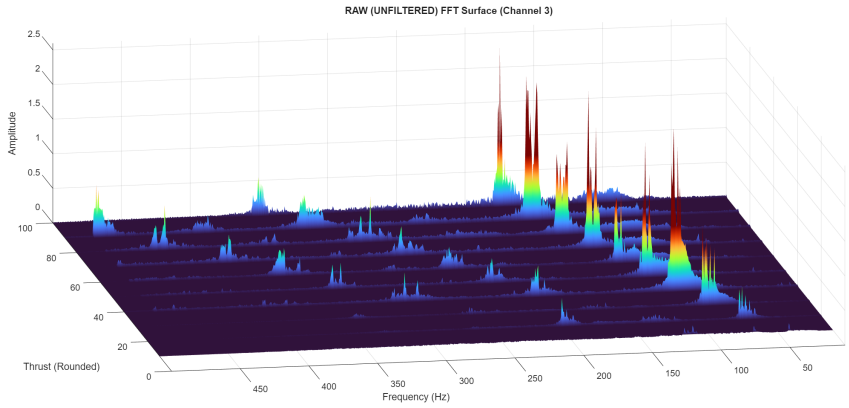


Figure 2.1: Fourier Transforms of IMU data during roll testing at different thrust levels

We then fit linear models to the first and second harmonic frequencies of this vibration and added it to the IMU models. We also have a basic magnetometer model, obtained by taking a reference magnetometer measurement in the earth frame, adding biases and multiplying by a 3x3 matrix of the form $I_{3\times3} + \delta B_{3\times3}$, where $\delta B_{3\times3}$ is a matrix of randomly generated small deviations, meant to model magnetic disturbances and miscalculation. We then rotate this into the body frame via the plant attitude quaternion, and add a white noise floor.

2.4.2 GPS

Our GPS sensor model follows a simple structure. our NEO-F9P module allows us to obtain centimeter-level position and very accurate velocity estimates via Doppler Shift + clock corrections from the base station. Our model then takes the true positions and velocities from the Plant outputs, and uses a Gauss-Markov Process to add noise to the system. The noise is modeled as:

$$\beta = e^{-\frac{t}{\tau}}$$
$$\epsilon_k = \beta \cdot \epsilon_{k-1} + \sqrt{1-\beta^2} \cdot \eta_{GPS}$$

Where η_{GPS} is a white noise process with covariance equal to the GPS estimated covariance (similar to the method used in the Wind Model). The velocity output is then obtained by running a complimentary filter between finite differences of GPS position values and the true plant velocity, and then adding more fresh white noise.

Chapter 3

Estimation and Filtering

3.1 Multiplicative Extended Kalman Filter [M-EKF]

3.1.1 Motivation

An aspect often overlooked by first timers working on controls for vertical take-off and landing vehicles is state estimation. In real life, sensors are not your friends. They will drift and lag, be noisy or susceptible to vibrations and temperature. Even then, there are some state proprieties which are very hard to directly measure, mainly attitude. One solution to the problem is to simply integrate the gyroscope data over time to obtain a positional estimate, and double-integrating the accelerometer data to obtain position estimates. Nevertheless, these options come with significant errors. A significant and unavoidable source of error comes from random walk accumulation from the sensor integration.
We first define estimates values for the noise densities of the accelerometers and the gyroscopes:

$$\rho_g = 3.57 \cdot 10^{-5}\ rad \cdot s^{-1}/\sqrt{Hz} \tag{3.1}$$

$$\rho_a = 4.02 \cdot 10^{-4}\ m \cdot s^{-2}/\sqrt{Hz} \tag{3.2}$$

We also note that for standard white noise, the expected standard deviation after time t is modeled by $\int_0^t \eta \cdot d\tau = \rho\sqrt{t}$. Thus, the standard deviation of the random walk produced by integrating the gyroscopes is $\sigma_{gyro} = \rho_g\sqrt{t}$. This result in itself is not too concerning. By looking at the error state dynamics (which will be derived and developed later), we can note that the acceleration error δa is coupled to the attitude error roughly by $\delta a = g \times \delta\theta$. From the previous step, we know that:

$$\delta\theta = \int_0^t \eta_g \cdot d\tau = \rho_g\sqrt{t} \tag{3.3}$$

From which:

$$\delta r = \int_0^t \int_0^t \delta a \, dx \, d\tau \tag{3.4}$$

$$\delta r = g \cdot \int_0^t \int_0^t \int_0^t \eta_g \, dy \, dx \, d\tau \tag{3.5}$$

$$\tag{3.6}$$

Thus, we can say that an approximate value for the standard deviation of the position error produced by the white noise on the gyroscopes is:

$$\sigma_{r,g} = g\rho_g \cdot \frac{t^{\frac{5}{2}}}{\sqrt{5}} \tag{3.7}$$

We also perform the integration on the accelerometers obtaining:

$$\sigma_{r,a} = \int_0^t \int_0^t \eta_a \, dx \, d\tau = \rho_a \cdot \frac{t^{\frac{3}{2}}}{\sqrt{3}} \tag{3.8}$$

We now combine both values as:

$$\sigma_r = \sqrt{\sigma_{r,g}^2 + \sigma_{r,a}^2} \tag{3.9}$$

Plugging in our values for noise densities, over an expected maximum flight time of 60 seconds, we find $\boldsymbol{\sigma_r = 4.38}$ meters. It is worth noting that this value is a best case scenario, as it fully ignores any sort of uncorrected sensor bias, any error accumulation from the forward Euler integrator used in the M-EKF, and it uses noise density values obtained under ideal conditions. Flight testing has demonstrated increased noise levels in the sensors. This clearly outlines the need for GPS backup for the position state estimate. Another reason to have an onboard M-EKF structure is to obtain initial state and bias estimates for the vehicle while on the ground. The 18-state filter structure we will present in this document allows for constant bias estimation under negligible acceleration conditions across all three sensors. It also makes for a robust sensor fusion method for the GPS data, while having the benefit of full dynamic observability of attitude states, removing filter stability concerns in the long term. This implementation is inspired by the work done by Matthew Hampsey on his M-EKF implementation [4], and expanded to include a GPS update step.

3.1.2 Error-State Formulation

Our error-state vector will be the following 18-dimensional vector denoted by δx:

$$\delta x = \begin{bmatrix} \alpha \\ \delta r \\ \delta v \\ \beta_\omega \\ \beta_f \\ \beta_m \end{bmatrix} \tag{3.10}$$

where α represents a small-angle error representation for our quaternion and is defined as:

$$\alpha = 2\delta q_v \tag{3.11}$$

The linearized dynamics for our states takes the following form:

$$\dot{\alpha} = -[\hat{\omega}]_\times \alpha - \beta_\omega - \eta_\omega \tag{3.12}$$
$$\delta\dot{r} = \delta v \tag{3.13}$$
$$\delta\dot{v} = -R_b^i(\hat{q})[a_m]_\times \alpha - R_b^i(\hat{q})\beta_f - R_b^i(\hat{q})\eta_f \tag{3.14}$$
$$\delta\dot{\beta}_\omega = \nu_\omega \tag{3.15}$$
$$\delta\dot{\beta}_f = \nu_f \tag{3.16}$$
$$\delta\dot{\beta}_m = \nu_m \tag{3.17}$$
$$\tag{3.18}$$

where a_m represents the measured acceleration from the accelerometer, and the ν terms are gaussian white noise processes. Derivations for the error state equations are presented here, adapted from Maley's work [5]:

Proof of 3.12:
We first define our quaternion as the product of a nominal quaternion with an error state quaternion:

$$q = \hat{q} \otimes \delta q \tag{3.19}$$
$$\delta q = \hat{q}^{-1} \otimes q \tag{3.20}$$

We then differentiate (3.20) w.r.t time. Applying the product rule yields:

$$\delta\dot{q} = \dot{\hat{q}}^{-1} \otimes q + \hat{q}^{-1} \otimes \dot{q} \tag{3.21}$$

We now note that since $\hat{q}^{-1} \otimes \hat{q} = 1$:

$$\frac{d(\hat{q}^{-1} \otimes \hat{q})}{dt} = \dot{\hat{q}}^{-1} \otimes \hat{q} + \hat{q}^{-1} \otimes \dot{\hat{q}} = 0 \tag{3.22}$$
$$\dot{\hat{q}}^{-1} = -\hat{q}^{-1} \otimes \dot{\hat{q}} \otimes \hat{q}^{-1} \tag{3.23}$$
$$\dot{\hat{q}}^{-1} = -\hat{q}^{-1} \otimes \frac{1}{2}\hat{q} \otimes \begin{bmatrix} 0 \\ \hat{\omega} \end{bmatrix} \otimes \hat{q}^{-1} \tag{3.24}$$
$$\dot{\hat{q}}^{-1} = -\frac{1}{2}\begin{bmatrix} 0 \\ \hat{\omega} \end{bmatrix} \otimes \hat{q}^{-1} \tag{3.25}$$

Replacing equations (3.25) and (2.8) into (3.21) yields, and using the fact that $\omega = \hat{\omega} + \delta\omega$ yields:

$$\delta\dot{q} = -\frac{1}{2}\begin{bmatrix}0\\ \hat{\omega}\end{bmatrix} \otimes \hat{q}^{-1} \otimes q + \hat{q}^{-1} \otimes \frac{1}{2} q \otimes \begin{bmatrix}0\\ \omega\end{bmatrix} \tag{3.26}$$

$$\delta\dot{q} = \frac{1}{2}\delta q \otimes \begin{bmatrix}0\\ \omega\end{bmatrix} - \frac{1}{2}\begin{bmatrix}0\\ \hat{\omega}\end{bmatrix} \otimes \delta q \tag{3.27}$$

$$\delta\dot{q} = \frac{1}{2}\delta q \otimes \begin{bmatrix}0\\ \hat{\omega}\end{bmatrix} - \frac{1}{2}\begin{bmatrix}0\\ \hat{\omega}\end{bmatrix} \otimes \delta q + \frac{1}{2}\delta q \otimes \begin{bmatrix}0\\ \delta\omega\end{bmatrix} \tag{3.28}$$

Since δq is an error quaternion, we can say $\delta q_1 \approx 1$. Applying the definition of quaternion multiplication further simplifies this to:

$$\delta\dot{q} = \frac{1}{2}\begin{bmatrix}-\delta q_v \cdot \hat{\omega}\\ \hat{\omega} + \delta q_v \times \hat{\omega}\end{bmatrix} + \frac{1}{2}\begin{bmatrix}\hat{\omega} \cdot \delta q_v\\ -\hat{\omega} - \hat{\omega} \times \delta q_v\end{bmatrix} + \frac{1}{2}\begin{bmatrix}-\delta q_v \cdot \delta\hat{\omega}\\ \delta\hat{\omega} + \delta q_v \times \delta\hat{\omega}\end{bmatrix} \tag{3.29}$$

Simplifying the vector components of the expression and neglecting second order terms finally produces:

$$\delta\dot{q}_v = -\hat{\omega} \times \delta q_v + \frac{1}{2}\delta\omega \tag{3.30}$$

Replacing in $\alpha = 2\delta q_v$ (Equation 3.11) and $\dot{\alpha} = 2\delta\dot{q}_v$, and also using the skew matrix form for cross products (written as $\left[\quad\right]_\times$):

$$\dot{\alpha} = -\left[\hat{\omega}\right]_\times \alpha + \delta\omega \tag{3.31}$$

Proof of 3.14:
This filter implementation assumes that the vehicle is undergoing no significant motion, thus the accelerometer measures a reference value of $\vec{g} = \begin{bmatrix}0\\0\\g\end{bmatrix}$ rotated into the body frame. The kinematic equation for velocity in the inertial frame is:

$$\dot{v} = R_b^i(q) \cdot a_{true} + \vec{g} \tag{3.32}$$

The accelerometer measurement includes bias and noise, so:

$$a_{true} = a_m - \beta_f - \eta_f \tag{3.33}$$

Replacing into 3.32:

$$\dot{v} = R_b^i(q) \cdot (a_m - \beta_f - \eta_f) + \vec{g} \tag{3.34}$$

The predicted velocity dynamics is a function of the estimated attitude and the accelerometer measurement:

$$\dot{\hat{v}} = R_b^i(\hat{q}) a_m + \vec{g} \tag{3.35}$$

We then define the error state of the inertial velocity as $\delta\dot{v} = \dot{v} - \dot{\hat{v}}$, which we substitute into the dynamics in 3.32:

$$\delta\dot{v} = [R_b^i(q) \cdot (a_m - \beta_f - \eta_f) + \vec{g}] - [R_b^i(\hat{q}) \cdot a_m + \vec{g}] \tag{3.36}$$
$$\delta\dot{v} = R_b^i(q) \cdot (a_m - \beta_f - \eta_f) - R_b^i(\hat{q}) \cdot a_m \tag{3.37}$$

It is now possible to show from Rodrigues' rotation formula that:

$$R_i^b(q) = 2q_v q_v^T + I_{3\times3}(q_w^2 - q_v^T q_v) - 2q_w[q_v]_\times \tag{3.38}$$
$$R_b^i(q) \approx R_b^i(\hat{q})(I + [\alpha]_\times) \tag{3.39}$$

Substituting this relationship into the error dynamics (and removing second order terms to linearize):

$$\delta\dot{v} = R_b^i(\hat{q})(I + [\alpha]_\times)(a_m - \beta_f - \eta_f) - R_b^i(\hat{q}) a_m \tag{3.40}$$
$$\delta\dot{v} = R_b^i(\hat{q}) a_m - R_b^i(\hat{q})\beta_f - R_b^i(\hat{q})\eta_f + R_b^i(\hat{q})[\alpha]_\times \cdot a_m - R_b^i(\hat{q}) a_m \tag{3.41}$$
$$\delta\dot{v} = R_b^i(\hat{q})[\alpha]_\times \cdot a_m - R_b^i(\hat{q})\beta_f - R_b^i(\hat{q})\eta_f \tag{3.42}$$
$$\delta\dot{v} = -R_b^i(\hat{q})[a_m]_\times \cdot \alpha - R_b^i(\hat{q})\beta_f - R_b^i(\hat{q})\eta_f \tag{3.43}$$

which proves the result.
Equations 3.12 to 3.18 lead to the following state-transition matrix in continuous time:

$$F = \begin{bmatrix} -[\hat{\omega}\times] & 0 & 0 & 0 & -I & 0 & 0 \\ 0 & 0 & I & 0 & 0 & -R_b^i(\hat{q}) & 0 \\ -R_b^i(\hat{q})[a_m\times] & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{3.44}$$

(3.45)

In an IMU update step, we have 9 measurements available to us. Accelerometer, Gyroscope, and Magnetometer. The gyroscopes are not used directly in the measurement update. So we will omit them from the measurement vector. These are packaged as follows:

$$z_k = \begin{bmatrix} a_m \\ B_m \end{bmatrix} \tag{3.46}$$

We then form our observation matrix H as follows (where $\vec{B}_i$ is the reference magnetometer measurement in the inertial frame):

$$H_{IMU} = \begin{bmatrix} \left[R_i^b(\hat{q}) \begin{pmatrix} 0 \\ 0 \\ g \end{pmatrix} \right]_\times & 0 & 0 & 0 & I & 0 \\ \left[R_i^b(\hat{q}) \cdot \vec{B}_i \right]_\times & 0 & 0 & 0 & 0 & I \end{bmatrix} \tag{3.47}$$

For a GPS-RTK update, we receive centimeter-level precision positional updates as well as high-accuracy velocity measurements, which we can then plug into our H matrix as:

$$z_k = \begin{bmatrix} r_m \\ v_m \end{bmatrix} \tag{3.48}$$

$$\tag{3.49}$$

$$H_{GPS} = \begin{bmatrix} 0 & I & 0 & 0 & 0 & 0 \\ 0 & 0 & I & 0 & 0 & 0 \end{bmatrix} \tag{3.50}$$

3.1.3 Multiplicative Extended Kalman Filter Formulation

In this section, we now lay out the formulation for our M-EKF, including an *a-priori* propagation step for the covariances and the states, as well as two *a-posteriori* update steps. One, a high-frequency update using our IMU measurements, and the second, a lower-frequency update using an GPS-RTK system.

Common Propagation Step

This step is common to both updates. We first begin by setting $\delta x = 0$ and propagating our nominal state $\hat{x} = x_{x|x-1}$ estimate using

$$x_{k|k-1} = f(x, u) \tag{3.51}$$

$$q_{k|k-1} = q_{k-1|k-1} + \dot{q}\Delta t \tag{3.52}$$

$$\dot{q} = \frac{1}{2} M(q) \begin{bmatrix} 0 \\ \omega \end{bmatrix} \tag{3.53}$$

in which

$$M(q) = \begin{bmatrix} q_0 & -q_1 & -q_2 & -q_3 \\ q_1 & q_0 & -q_3 & q_2 \\ q_2 & q_3 & q_0 & -q_1 \\ q_3 & q_2 & q_1 & q_0 \end{bmatrix} \tag{3.54}$$

and $f(x, u)$ represents our non-linear dynamics function. We can then discretize our state transition matrix using a second-order pade approximation of

$$\Phi_k = e^{F\Delta t} \tag{3.55}$$

The *a-priori* estimate covariance is:

$$P_{k|k-1} = \Phi P_{k-1|k-1}\Phi_k^T + Q_k \tag{3.56}$$
$$\tag{3.57}$$

IMU Update Step

The Kalman gain L_k is as follows:

$$L_k = P_{k|k-1}H_{_{IMU}}^T(H_{_{IMU}}P_{k|k-1}H_{_{IMU}}^T + R_{_{IMU}})^{-1} \tag{3.58}$$

$R_{_{IMU}}$ is the measurement covariance matrix.

$$R_{_{IMU}} = \begin{bmatrix} \sigma_a^2 & 0 \\ 0 & \sigma_{_B}^2 \end{bmatrix} \tag{3.59}$$

We then update the *a-posteriori* error-state and covariance estimate using the Joseph form.

$$\delta x = L_k(z_k - \hat{z_k}) \tag{3.60}$$
$$P_{k|k} = (I - L_kH_{_{IMU}})P_{k|k-1}(I - L_kH_{_{IMU}})^T + L_kR_{_{IMU}}L_k^T \tag{3.61}$$

We then update the *a-posteriori* quaternion estimate by

$$q_{k|k} = q_{k|k-1} \otimes \delta q(\alpha) \tag{3.62}$$
$$\tag{3.63}$$

where $\delta q(\alpha) = \begin{pmatrix} 1 \\ \frac{\alpha}{2} \end{pmatrix}$. We then update the rest of the state additively via:

$$x_{k|k} = x_{k|k-1} + \delta x \tag{3.64}$$

we then reset $\delta x = 0$ and proceed.

GPS Update Step

If there is a new GPS-RTK packet available, this update step takes place. The Kalman gain L_k is as follows:

$$L_k = P_{k|k-1}H_{_{GPS}}^T(H_{_{GPS}}P_{k|k-1}H_{_{GPS}}^T + R_{_{GPS}})^{-1} \tag{3.65}$$

We then update the *a-posteriori* error-state and covariance estimate using the Joseph form.

$$\delta x = L_k(z_k - \hat{z_k}) \tag{3.66}$$

$$P_{k|k} = (I - L_k H_{GPS})P_{k|k-1}(I - L_k H_{GPS})^T + L_k R_{GPS} L_k^T \tag{3.67}$$

The *a-posteriori* update step is identical to that of the IMU, where we also reset the error state to zero at the end.

3.1.4 Onboard Application

For practical purposes, the M-EKF implementation in our onboard flight computer differs from the one presented here. In flight, we use two different estimator modes, our "Ground Estimator" which is the full 18-state filter presented above, and the "Flight Estimator", which skips the IMU Update step and operates with a reduced state vector without the bias states. Right before the phase of flight is changed from grounded to in-flight, the filter mode switches to flight, and uses a warm start sequence, by using the last available covariance matrix $P_{k|k}$ from the Ground Estimator function. There are a few motivations behind this design choice, mainly computational efficiency, the input feed-through from the thrust directly into the accelerometer causing stability issues (which can't be accounted for due to the lack of encoder feedback on the gimbal actuators), and to further simplify the system, as magnetic disturbances have been observed at higher levels during flight (likely due to the current draw form the propellers), making the sensor cause trouble.
We can show how this reduction to a 9 dimensional state vector from an 18 dimensional state vector is a significant performance gain in flight by looking at the number of floating point operations (FLOPs) required to perform one MEKF step on both filters.
We first note that to multiply an $n \times m$ matrix with an $m \times p$ matrix requires a total of $\boldsymbol{2abc - ac}$ FLOPs. Applying this formula to our filter, we can find the number of FLOPs required for one pass of the Ground Estimator vs. one pass of the Flight Estimator:
As is shown in Table 3.1, the Ground Estimator mode requires **9.7** times the number of FLOPs as

FLOPs per stage on both filter modes		
Filter Stage	Ground Estimator	Flight Estimator
Propagation	22,680	2,754
IMU Update	37,224	N/A
GPS Update	37,224	7,254
Total	**97,128**	**10,008**

Table 3.1: Filter Mode Comparison

the Flight Estimator, and this is ignoring the cost of matrix inversion in the Kalman Gain update steps, which is also increasingly expensive for larger matrices.

3.1.5 Observability Analysis

In this section, we seek to analyze the observability proprieties of both estimator modes, starting with the Ground Estimator.

Ground Estimator Observability:
We first define the observability matrix $\mathcal{O}$ for our system as:

$$\mathcal{O} = \begin{bmatrix} HF \\ HF^2 \\ HF^3 \\ \vdots \\ HF^{n-1} \end{bmatrix} \tag{3.68}$$

For a system to be fully observable, the rank of this observability matrix needs to equal the state vector dimension. $\text{rank}(\mathcal{O}) = n$. Evaluating the rank when the vehicle is still on the ground produces a rank of **17**, indicating we have one unobservable state. This state is one of the magnetometer bias states, and it comes from the magnetometer observability equation:

$$z_{mag} = [B^b]_\times \cdot \alpha + I\beta_m \tag{3.69}$$

Which, after expanding the cross product, produces 4 unknowns and only 3 equations (this equation is obtained by looking at the magnetometer rows for the product $H\delta x$). The unobservable state is thus one of the magnetometer bias states. It is possible to prove that the system is dynamically observable by analyzing it's local observability as a Linear Time Varying (LTV) system. [7] For an LTV system, we define the local observability matrix $\mathcal{O}_l$ over the interval $[t_k, t_{k+j-1}]$ as:

$$\mathcal{O}_l = \begin{bmatrix} H_k \\ H_{k+1}\Phi_k \\ H_{k+2}\Phi_{k+1}\Phi_k \\ \vdots \\ H_{k+j-1}\Phi_{k+j-2}\cdots\Phi_k \end{bmatrix} \tag{3.70}$$

Note the use of the discrete state transition matrix Φ instead of the continuous version. The observability criterion for the LTV system is also different, as the LTV system is considered to be observable if $\mathbf{rank(\mathcal{O}_l^T \mathcal{O}_l) = n}$.
Consider a reduced system with state vector containing only the attitude error state and the magnetometer bias. We define H as:

$$H_k = \begin{bmatrix} [\vec{B}_k^b]_\times & I \end{bmatrix} \tag{3.71}$$

We also observe that the block of F corresponding to the attitude states is dominated by the angular rates. If the chosen timestep between time steps is small, we can reasonably say that $\Phi = I_{6\times6}$. So our local observability matrix between t_k and t_{k+1} takes the form:

$$\mathcal{O}_l = \begin{bmatrix} [\vec{B}_k^b]_\times & I_{3\times3} \\ [\vec{B}_{k+1}^b]_\times & I_{3\times3} \end{bmatrix} \tag{3.72}$$

We now prove the system is observable by contradiction. Assume the system is unobservable even with rotation; this implies the existence of a non-trivial vector $v = \begin{bmatrix} \alpha \\ \beta_m \end{bmatrix}$ in the null-space of $\mathcal{O}_l$. The condition $\mathcal{O}_l v = 0$ expands to the following simultaneous equations for time steps k and $k+1$:

$$[\vec{B}_k^b]_\times \alpha + \beta_m = 0 \quad \Rightarrow \quad \beta_m = -[\vec{B}_k^b]_\times \alpha \tag{3.73}$$

$$[\vec{B}_{k+1}^b]_\times \alpha + \beta_m = 0 \quad \Rightarrow \quad \beta_m = -[\vec{B}_{k+1}^b]_\times \alpha \tag{3.74}$$

Recall the property of the cross product matrix $[u]_\times v = u \times v$. The result of a cross product is always perpendicular to the constituent vectors. Therefore, Equation (1) and Equation (2) impose rigid geometric constraints on the bias error β_m:

1. From Eq. (3.73), $\beta_m = \alpha \times \vec{B}_k^b$. This implies $\beta_m \perp \vec{B}_k^b$.
2. From Eq. (3.74), $\beta_m = \alpha \times \vec{B}_{k+1}^b$. This implies $\beta_m \perp \vec{B}_{k+1}^b$.

If the vehicle undergoes a rotation such that the magnetic field vector changes direction in the body frame ($\vec{B}_k^b \not\parallel \vec{B}_{k+1}^b$), the two vectors define a plane. The only vector direction that can be simultaneously perpendicular to two non-parallel vectors is their cross product. Thus, β_m must be parallel to the normal of this plane:

$$\beta_m \parallel (\vec{B}_k^b \times \vec{B}_{k+1}^b) \tag{3.75}$$

However, β_m is a constant sensor bias. For a general rotation (where the axis of rotation does not align perfectly with the magnetic field line), the normal vector $(\vec{B}_k^b \times \vec{B}_{k+1}^b)$ changes orientation relative to the inertial frame constraints of the attitude error α.
The only solution that respects the geometric constraints that β_m must be constant, yet always perpendicular to the changing $\vec{B}^b$ vector forces the magnitude of the error to zero:

$$\beta_m = 0 \implies \alpha = 0 \tag{3.76}$$

This confirms that the null-space is empty for generic rotations, and by extension, restoring the rank of the full system to $n = 18$.

Flight Estimator Observability:
For the Flight Estimator, the state vector is reduced to $n = 9$, consisting only of attitude error, position error, and velocity error: $\delta x = [\alpha^T, \delta r^T, \delta v^T]^T$. The reduced system ignores sensor biases. The measurement vector consists of GPS position and velocity $z = [r_m^T, v_m^T]^T$.
We define the linearized measurement matrix H and the relevant blocks of the state transition matrix F (ignoring small angular rate terms for clarity) as:

$$H = \begin{bmatrix} 0_{3\times3} & I_{3\times3} & 0_{3\times3} \\ 0_{3\times3} & 0_{3\times3} & I_{3\times3} \end{bmatrix}, \quad F \approx \begin{bmatrix} 0_{3\times3} & 0_{3\times3} & 0_{3\times3} \\ 0_{3\times3} & 0_{3\times3} & I_{3\times3} \\ -R[a_m]_\times & 0_{3\times3} & 0_{3\times3} \end{bmatrix} \tag{3.77}$$

We construct the observability matrix $\mathcal{O}$ and evaluate it's rank to be 8, giving us one unobservable state which corresponds to the null space of $[a_m]_\times$, which in this flight condition is the Z-axis. This confirms that **Yaw** (ψ) is unobservable from GPS measurements alone during vertical flight. Physically, a yaw rotation about the gravity vector produces no change in the acceleration vector in the inertial frame, rendering it invisible to the velocity updates.
However, similar to the Ground Estimator, the system becomes fully observable under dynamic conditions. If the vehicle undergoes **lateral acceleration**, the specific force vector a_m tilts away from the vertical axis. If the flight trajectory includes changes in the acceleration direction such that $a_m(t_k)$ is not parallel to $a_m(t_{k+1})$, the intersection of the null spaces vanishes, and the rank is restored to $n = 9$.

3.2 Digital Filter Sequence [DFS]

3.2.1 Motivation

As was discussed in the IMU Models section of this paper, strong vibrational spikes were seen during ground testing of the vehicle. Since those tests, additional mechanical isolation was added for the flight computer, but a digital filter sequence was added anyways to prevent this issue. The digital filter is made up of two stages: First, an infinite impulse response (IIR) notch filter with dynamic cutoff frequency is designed to absorb the noise from the vibrational spikes by leveraging the fact that the frequency vs. commanded thrust relationship is sufficiently linear. Then, a second order Butterworth filter is designed to kill the higher frequency harmonics and help reduced the increased white noise floor that was seen. This specific combination was chosen to minimize complexity, and maximize attenuation while minimizing phase impact to the 0-5Hz range, which is where we could expect our controls to operate. The magnitude and phase response of an n-th order Butterworth filter is given by:

$$|H(f)| = \frac{1}{\sqrt{1 + \left(\frac{f}{f_c}\right)^{2n}}} \qquad \Phi(f) = -\arctan\left(\frac{\sqrt{2}\cdot\frac{f}{f_c}}{1-\left(\frac{f}{f_c}\right)^2}\right) \tag{3.78}$$

ASTRAv2 spends most of the flight at 70% thrust, which has a vibrational spike of around 140Hz. For a filter with $f_c = 30Hz$, $\boldsymbol{|H(140)| = 0.046}$ and $\boldsymbol{\Phi(140) = -13.63\,\text{deg}}$. This indicates that the filter would absorb 95.4% of the vibrations. For a standard control loop with around 45 degrees of phase margin, we can afford -13.63 degrees of added phase without muck losing greatly on margins. Nevertheless, the leftover 6% of noise might be troubling considering the magnitude of the spikes seen. Figure 3.1 shows a sample accelerometer measurement from a thrust test at 50% thrust.
since the vibrational spikes have been consistent across dozens of tests, and their width does not seem too distributed, it is a prime application for a Notch filter to surgically remove the remaining noise, with minimal phase effects outside it's intended target.

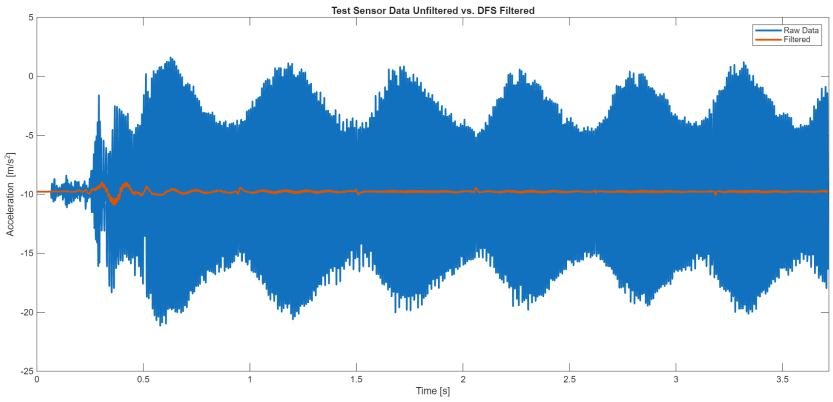


Figure 3.1: Sample Accelerometer data from Thrust Testing

3.2.2 Analog and Discrete Formulation

Let us first start with the process for the 2nd order Butterworth filter. The standard transfer function in the s-domain for this filter takes the form:

$$H(s) = \frac{\omega_c^2}{s^2 + \sqrt{2}\omega_c s + \omega_c^2} \tag{3.79}$$

where ω_c is the filter's chosen cutoff frequency. We first need to discretize this filter by transforming into the z-domain via the bilinear transform. We define a pre-warping gain C and use a normalized filter transfer function:

$$H(s) = \frac{1}{s^2 + \sqrt{2}s + 1} \qquad\qquad C = \tan\left(\frac{\omega_c T}{2}\right) \tag{3.80}$$

where T is the time step between filter calls. Substituting $s = \frac{1}{C}\frac{z-1}{z+1}$:

$$H(z) = \frac{1}{\left(\frac{1}{C}\frac{z-1}{z+1}\right)^2 + \sqrt{2}\left(\frac{1}{C}\frac{z-1}{z+1}\right) + 1} \tag{3.81}$$

$$H(z) = \frac{C^2(z+1)^2}{(z-1)^2 + \sqrt{2}C(z-1)(z+1) + C^2(z+1)^2} \tag{3.82}$$

$$H(z) = \frac{C^2(z^2+2z+1)}{(1+\sqrt{2}C+C^2)z^2 + (2C^2-2)z + (1-\sqrt{2}C+C^2)} \tag{3.83}$$

$$H(z) = \frac{C^2(1+2z^{-1}+z^{-2})}{(1+\sqrt{2}C+C^2) + (2C^2-2)z^{-1} + (1-\sqrt{2}C+C^2)z^{-2}} \tag{3.84}$$

We now seek to find a difference equation of the form:
$y[n] = b_0x[n] + b_1x[n-1] + b_2[n-2] - a_1y[n-1] - a_2y[n-2]$ for the filter. We do this by noting that

$H(z) = \frac{Y(z)}{X(z)}$, and that $Y(z)z^{z-1} = y[n-1]$. Multiplying inputs and numerator and outputs with denominator, and dividing such that $a_0 = 1$ yields the following coefficients: $D = 1 + \sqrt{2}C + C^2$

$$b_0 = \frac{C^2}{D} \qquad b_1 = 2b_0 \qquad b_2 = b_0 \tag{3.85}$$

$$a_0 = 1 \qquad a_1 = \frac{2C^2 - 2}{D} \qquad a_2 = \frac{1 - \sqrt{2}C + C^2}{D} \tag{3.86}$$

To discretize the notch filter, we use a different method [3] by directly mapping the filter poles and zeros to the digital domain. The digital transfer function for a notch filter is:

$$H(z) = G\frac{1 - 2\cos(\omega_0 T)z^{-1} + z^{-2}}{1 - 2r\cos(\omega_0 T)z^{-1} + r^2 z^{-2}} \tag{3.87}$$

$$\tag{3.88}$$

$$r = e^{-\pi \frac{Q}{f_s}} \tag{3.89}$$

where f_s is the sampling frequency and Q is the notch width. The full derivation for the digital transfer can be found in Hasegawa's lecture on Notch Filters [3]. We now find the normalization gain G such that $H(1) = 1$ to be:

$$G = \frac{1 - 2r\cos(\omega_0 T) + r^2}{2 - 2\cos(\omega_0 T)} \tag{3.90}$$

We can now implement the difference equation:

$$y[n] = G(x[n] - 2\cos(\omega_0 T)x[n-1] + x[n-2]) + 2r\cos(\omega_0 T)y[n-1] - r^2 y[n-2] \tag{3.91}$$

To apply both these filters consecutively, we just feed the outputs of the notch as inputs to the Butterworth filter. An important consideration for notches is making sure the poles are within the unit circle with some margin for feedback stability ($r < 1$). This condition is satisfied in our case as the notch has a width of around 25Hz in our filter to fully capture the spikes.

Figure 3.2 shows a heatmap version of a Bode Plot for our digital filter sequence, where the dynamic notch frequency is clearly visible.

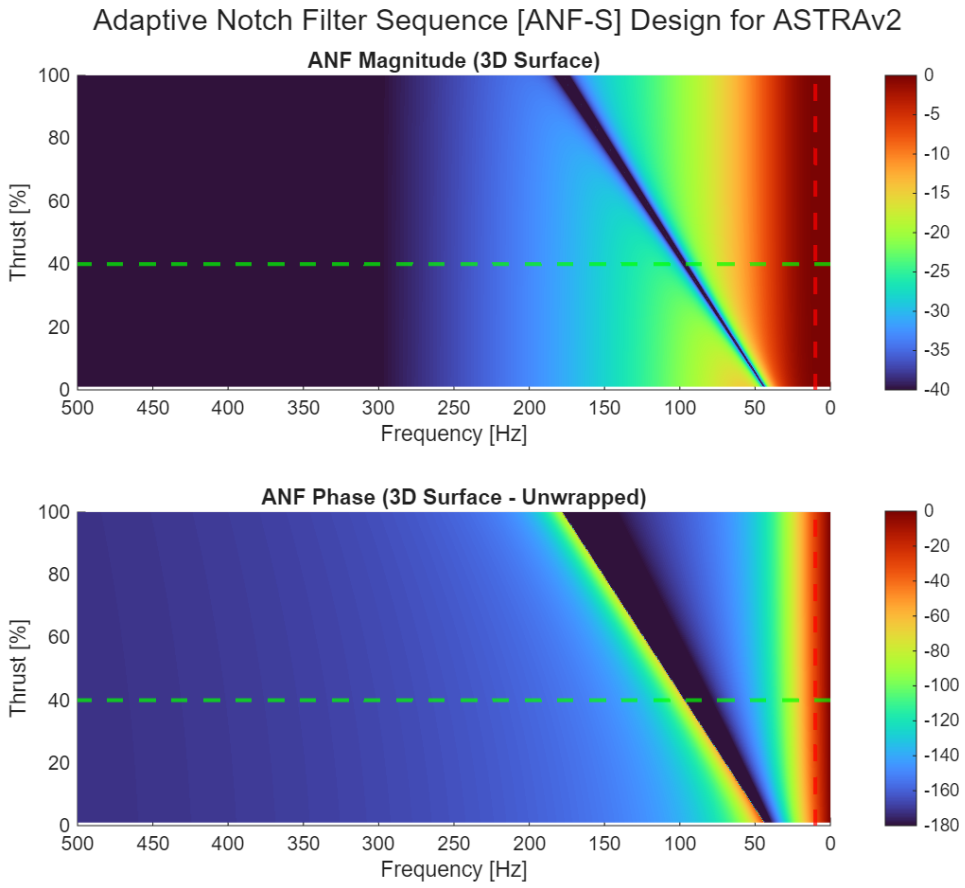


Figure 3.2: Bode Plot for the Digital Filter Sequence

Chapter 4

Controller

4.1 Controller Iteration History

Ironically, the vehicle tracking controller ended up being one of the most overlooked items early in the development history for ASTRAv2. The initial controller solution was a Linear Quadratic Regulator (LQR) meant to track a reference trajectory generated via a Genetic Algorithm iterating over multiple cost matrix pairs until it generated a solution that minimized certain optimality metrics. Methods using Sequential Convexification to solve constrained optimization problems were also floated (and even implemented by a partner team) to generate trajectories for this tracking controller. Nevertheless, in an effort to reduce initial implementation complexity, make for a smoother on-ramp for new team members, and since the CPLC Milestone does not require anything more complex than a simple hover, these methods were replaced by a simple checkpoint system that switches checkpoints either at a certain time, or when a condition is met (5 seconds within $1m^3$ for example), whichever comes first. The LQR solution however was kept until the first round of hover tests, until the controller was adapted to a 3-stage cascaded controller in an effort to obtain more tuning parameters when flight tests revealed we'd experience more actuator and sensor error than desired.

4.2 Control Law Formulation

The control law used for ASTRAv2 is composed of a three-stage cascaded controller with 2 PID loops and one LQR controller stage for attitude tracking, which is augmented with integral action (LQRi). It's worth noting that this stage acts as another PID loop if the gain matrix is just pre-computed and one does not re-linearize dynamics in-flight. We've kept it as an LQR because of that possible upgrade if it is deemed needed, which is a very attractive method for gain scheduling. Figure 4.1 shows a simplified block diagram of the controller's structure. It's worth noting that the target position vector is simply the current checkpoint's position vector. One of the big benefits of having a cascaded structure is the ability to get in-between steps easily to add clamping, integral anti-windup and soft-gating. It also gives the controls engineer more adjustable parameters, moving from two cost matrices with indirect effect on the actual control gain matrix, it removes a layer of abstraction and allows direct tuning of the gains, making the testing campaign more

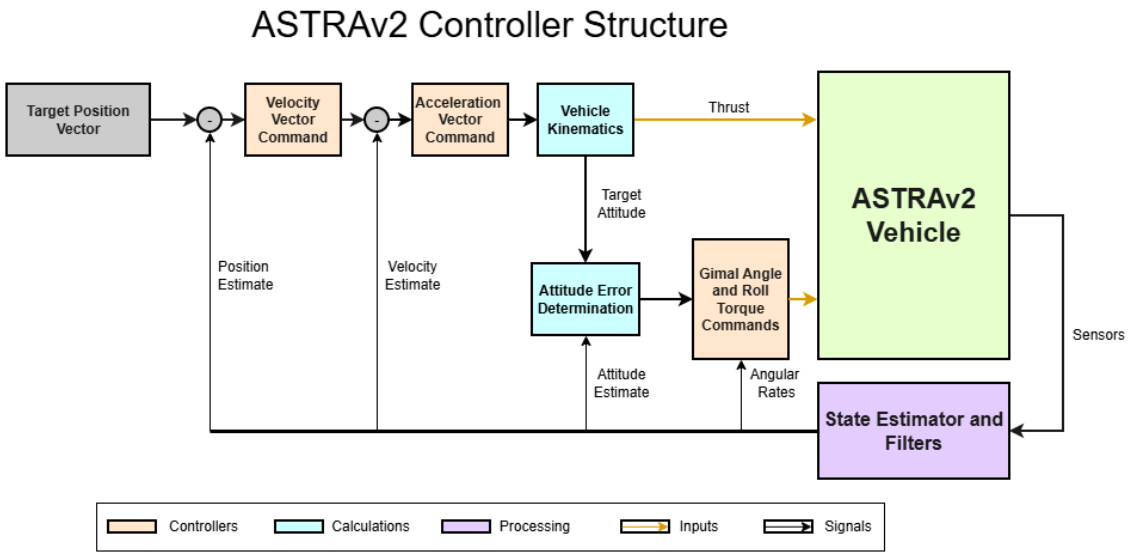


Figure 4.1: ASTRAv2 Controller Block Diagram

efficient and effective.

Position Control Loop:
The first controller stage is a simple proportional loop in charge of emitting a target velocity vector. We will use $\varepsilon = x_{trg} - x$ to denote the errors:

$$\vec{v}_{trg} = \vec{K}_p \cdot \vec{\varepsilon}_{pos} \tag{4.1}$$
$$\vec{v}_{trg} = \textbf{clamp}(\vec{v}_{trg},\ \vec{c}_v) \tag{4.2}$$

where $\vec{c}_v$ is a 2×3 vector of min and max values for the vector, and **clamp()** simply saturates the vector element-wise with those values.

Velocity Control Loop:
The second controller stage is a PI control loop in charge of emitting a target acceleration vector $\vec{a}_{trg}$: The integrator for this stage uses a soft-gating method dependent on the square of the attitude errors and the velocity errors themselves:

$$\vec{a}_{trg} = \vec{K}_a \cdot \vec{\varepsilon}_{vel} + \vec{K}_i \cdot \vec{I}_{vel} \tag{4.3}$$
$$\vec{a}_{trg} = \textbf{clamp}(\vec{v}_{trg},\ \vec{c}_v) \tag{4.4}$$

Note that the target acceleration clamp values are chosen such that the maximum vertical tilt of the vehicle never exceeds 15-20 degrees. The soft gating method takes the following form, using a gaussian form so that when the error grows far beyond the threshold at the current timestep, the

integral stops accumulating:

$$\vec{\kappa} = \sqrt{\frac{\varepsilon_{att}^2}{\varepsilon_{MAX}^{att}} + \frac{\varepsilon_{vel}^2}{\varepsilon_{MAX}^{vel}}} \tag{4.5}$$

$$\vec{I}_{vel} = \int_0^{t_k} e^{-\vec{\kappa}^2} \cdot \varepsilon_{vel} \; d\tau \tag{4.6}$$

$$\vec{I}_{vel} = \mathbf{clamp}(\vec{I}_{vel}, \; \vec{c}_{I,vel}) \tag{4.7}$$

This method is paired with a standard clamp to prevent integral windup, which in essence makes the controller not overreact to downstream errors, making it focus only on correcting steady state errors, while keeping the benefits of having a more aggressive gain.

Kinematics Step:
We first need to convert this target acceleration vector into an attitude quaternion for our final attitude control loop. We first turn this acceleration target into a force target, and set our thrust input as:

$$\vec{F}_{trg} = m\vec{a}_{trg} \tag{4.8}$$

$$T = ||\vec{F}_{trg} + \vec{g}|| \tag{4.9}$$

We then take that force vector, and clamp it such that it's z component can never be lower than our minimum thrust. This is done to make sure the target attitude does not flip upside down when the vehicle needs to accelerate downward. We seek to align the vehicle's z-axis such that it points parallel to our target acceleration vector. We accomplish this by creating a rotation matrix representing this orientation, and then using a standard algorithm to transform this to an attitude quaternion. We first have to establish a target heading reference, which for simplicity we define as a vector pointing north: $\vec{\psi} = [1, 0, 0]^T$, we can now generate the Direction Cosine Matrix (DCM) by:

$$\vec{Z}_b = \frac{\vec{a}_{trg}}{||\vec{a}_{trg}||} \tag{4.10}$$

$$\vec{Y}_b = \vec{Z}_b \times \vec{\psi} \tag{4.11}$$

$$\vec{X}_b = \frac{\vec{Y}_b \times \vec{Z}_b}{||\vec{Y}_b||} \tag{4.12}$$

$$R_{trg} = \begin{bmatrix} \vec{X}_b & \vec{Y}_b & \vec{Z}_b \end{bmatrix} \tag{4.13}$$

We can now convert this to a quaternion target using Shepperd's method. Note that we can use this simple version and not the robust version since we've guaranteed that the vehicle is always

upright with our clamping in earlier steps:

$$\mathbf{tr}(R_{trg}) = R_{11} + R_{22} + R_{33} \tag{4.14}$$

$$q_0 = \frac{1}{2}\sqrt{1 + \mathbf{tr}(R_{trg})} \tag{4.15}$$

$$q_1 = \frac{R_{32} - R_{23}}{4q_0} \tag{4.16}$$

$$q_2 = \frac{R_{13} - R_{31}}{4q_0} \tag{4.17}$$

$$q_3 = \frac{R_{21} - R_{12}}{4q_0} \tag{4.18}$$

$$\mathbf{q}_{trg} = \begin{bmatrix} q_0 & q_1 & q_2 & q_3 \end{bmatrix}^T \tag{4.19}$$

Attitude Control Loop (LQRi):
This section will cover the gain matrix generation process for our LQRi attitude control law. We will first start by linearizing our dynamics and writing them in state-space representation.
Let $\dot{x} = f(x, u)$. We seek to write our attitude dynamics as:

$$\dot{\boldsymbol{x}} = A\boldsymbol{x} + B\boldsymbol{u} \tag{4.20}$$

$$x = [q^T,\ \omega^T]^T \tag{4.21}$$

$$u = [\theta,\ \varphi,\ \tau_{roll}]^T \tag{4.22}$$

We can now linearize said dynamics by finding the Jacobian matrices

$$A = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \qquad B = \begin{bmatrix} \frac{\partial f_1}{\partial u_1} & \cdots & \frac{\partial f_1}{\partial u_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial u_1} & \cdots & \frac{\partial f_n}{\partial u_m} \end{bmatrix} \tag{4.23}$$

and evaluating them at our critical points:

$$x_c = \begin{bmatrix} \mathbf{0}_{7\times 1} \end{bmatrix}$$

$$u_c = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$$

This full 7-state system is technically uncontrollable, as our inputs have no effect on q_w, the scalar component of the quaternion. This isn't problematic however, as it is meant to maintain the unit constraint that $\sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2} = 1$. We can thus map this state down using a transformation matrix T such that:

$$T = \begin{bmatrix} \mathbf{0_{6\times 1}} \\ \boldsymbol{I_{6\times 6}} \end{bmatrix} \tag{4.24}$$

We can now define our control matrices A_c and B_c using T^+ as the Moore-Penrose inverse for T:

$$A_c = T^+ A T \qquad B_c = T^+ B \tag{4.25}$$

To add the integral action states, we can now augment the system as:

$$A_{aug} = \begin{bmatrix} A_c & \mathbf{0_{6\times3}} \\ \boldsymbol{-I_{3x3}} & \mathbf{0_{3\times6}} \end{bmatrix} \qquad B_{aug} = \begin{bmatrix} B_c & \mathbf{0_{3\times3}} \end{bmatrix} \tag{4.26}$$

We can now use this to solve a continuous time (or discrete time) algebraic Ricatti equation to find the optimal gain matrix K for our control law of the form $\boldsymbol{u = -Kx}$. The process to solve this infinite horizon LQR problem is detailed in Boyd's Lecture notes [1], but it involves designing two square cost matrices Q & R which penalize state error and actuator effort, respectively. Once we have obtained our gain matrix K_{att} (which for now, is precomputed before flight and stored), we have to first define the attitude error, because when dealing with quaternions, we cannot define the error as $x_{trg} - x$. We instead define the error multiplicatively:

$$q \otimes \delta q = q_{trg} \tag{4.27}$$

$$\delta q = q^{-1} \otimes q_{trg} \tag{4.28}$$

$$\delta q = q^{*} \otimes q_{trg} \tag{4.29}$$

Where δq represents the difference quaternion (which we use as the error), and q^* is the quaternion conjugate. We now negate this result if the scalar part is negative (to ensure shortest rotation possible) and define the attitude error $\boldsymbol{\varepsilon_{att} = [\delta q_x,\ \delta q_y,\ \delta q_z]^T}$ since as discussed earlier, we can only control the vector states of the quaternion. Our remaining inputs are then calculated by:

$$\begin{bmatrix} \theta \\ \varphi \\ \tau_{roll} \end{bmatrix} = K_{att} \begin{bmatrix} -\epsilon_{att} \\ \omega \\ I_{att} \end{bmatrix} \tag{4.30}$$

I_{att} is our integral accumulator for the attitude error, which is a simple integral accumulator with a clamp to prevent windup:

$$\vec{I}_{att} = \int_0^{t_k} \varepsilon_{att}\ d\tau \tag{4.31}$$

$$\vec{I}_{att} = \mathbf{clamp}(\vec{I}_{att},\ \vec{c}_{I,att}) \tag{4.32}$$

Chapter 5

Simulation Outputs

The following plots were extracted from a Simulation run with wind speeds of up to $4ft \cdot s^{-1}$, a considerable CoM offset, parameter errors of $\pm 25\%$ sampled randomly before flight, gimbal misalignment of up to a degree, and the complete set of noise and disturbance models presented.

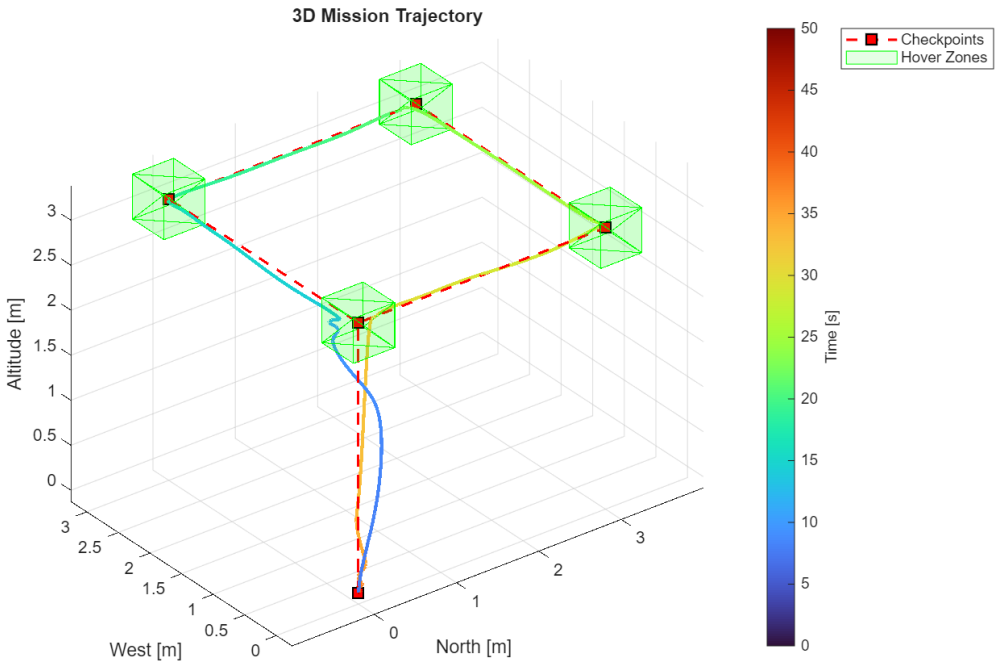


Figure 5.1: Simulated Flight Trajectory

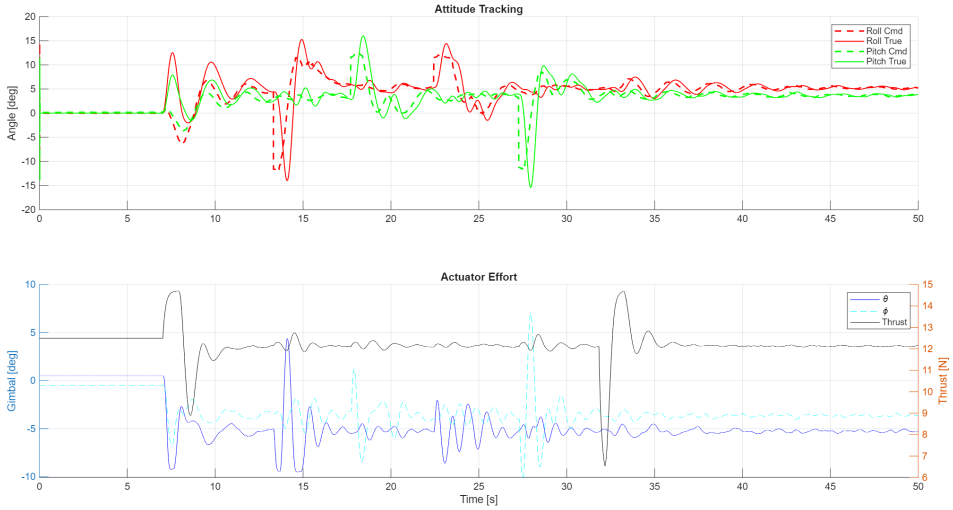


Figure 5.2: ASTRAv2 Simulated Actuator Effort

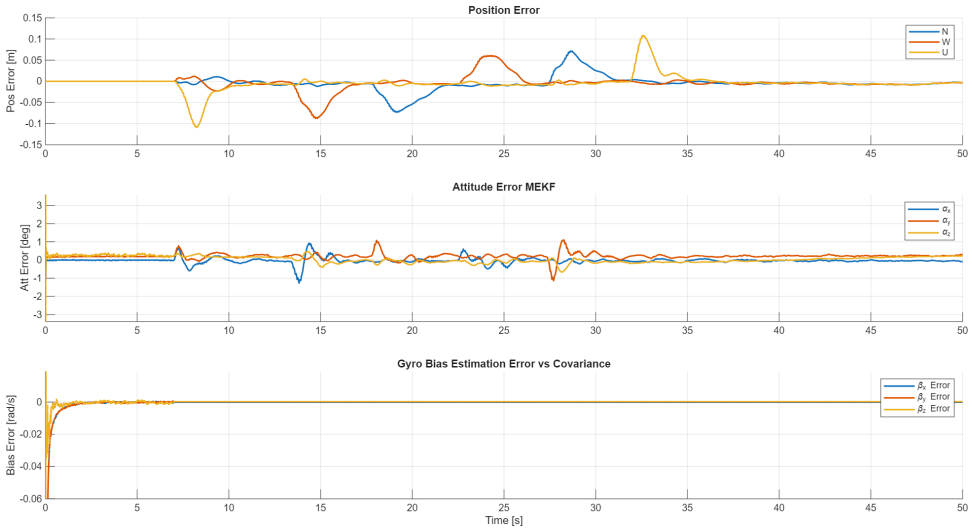


Figure 5.3: M-EKF Filter Performance

Bibliography

[1] Boyd, S. (2009). *Continuous-time LQR* [Lecture notes]. EE363: Linear Dynamical Systems, Stanford University. https://stanford.edu/class/ee363/lectures/clqr.pdf

[2] Yang, Y. (2013). Quaternion-based LQR spacecraft control design is a robust pole assignment design. *Journal of Aerospace Engineering*, *26*(3), 600–607. https://doi.org/10.1061/(ASCE)AS.1943-5525.0000232

[3] Hasegawa-Johnson, M. (2020). *Lecture 14: Notch Filters* [PowerPoint slides]. Department of Electrical and Computer Engineering, University of Illinois. https://courses.physics.illinois.edu/ece401/fa2020/slides/lec14.pdf

[4] Hampsey, M. (2020, July 18). *The Multiplicative Extended Kalman Filter*. https://matthewhampsey.github.io/blog/2020/07/18/mekf

[5] Maley, J. M. (2013). *Multiplicative quaternion extended Kalman filtering for nonspinning guided projectiles* (Report No. ARL-TR-6503). U.S. Army Research Laboratory.

[6] Brunton, S. (2017). *Control Bootcamp* [YouTube playlist]. YouTube. https://www.youtube.com/playlist?list=PLMrJAkhIeNNR20Mz-VpzgfQs5zrYi085m

[7] Shen, K., Proletarsky, A. V., & Neusypin, K. A. (2016). Quantitative Analysis of Observability in Linear Time-Varying Systems. *Proceedings of the 35th Chinese Control Conference* (pp. 44–48). Chengdu, China. https://ieeexplore.ieee.org/document/7553058